UNIVERSITI PUTRA MALAYSIA

BERILMU BERBAKTI

FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

DEPARTMENT OF COMMUNICATION TECHNOLOGY AND NETWORK

SEMESTER II 2025/2026

COURSE : BACHELOR OF COMPUTER SCIENCE (COMPUTER NETWORK) WITH HONOURS

COURSE CODE : CSC4202 (DESIGN AND ANALYSIS OF ALGORITHMS)

LECTURE GROUP : GROUP 2

PROJECT TITLE : WILDFIRE EVACUATION ROUTE PLANNING USING DIJKSTRA'S SHORTEST PATH ALGORITHM

LECTURER : DR NUR ARZILAWATI MD YUNUS

Name	Matric Number
SUJENIZSWARAN A/L RAJAH	225284
NUFAIL AZRI BIN AZMI	224872
DZAKY KEENAN IBRAHIM	218430
MUHAMMAD AZIB BIN AZMI	223796

Table of Contents

1.0 Introduction

Wildfires are among the most destructive natural disasters, posing significant threats to human lives, property, and the environment. During a wildfire emergency, road closures and rapidly changing conditions can make evacuation difficult, increasing the risk to affected communities. Therefore, efficient evacuation planning is essential to ensure that residents can reach designated shelters safely and in the shortest possible time.

This project presents a wildfire evacuation route planning system for the fictional village of Cedar Hollow. The village road network is modelled as a weighted graph, where each location is represented as a vertex and each road segment is represented as an edge with an associated travel distance. Roads blocked by the wildfire are excluded from the network to ensure that only safe routes are considered during route planning.

To solve this problem, Dijkstra's Algorithm is selected because it efficiently computes the shortest path in weighted graphs with non-negative edge weights. The algorithm determines the minimum travel distance from a selected source location to the nearest reachable evacuation shelter while avoiding blocked roads. A Java-based application is developed to implement the proposed solution and demonstrate how the algorithm can assist emergency evacuation planning.

The project also evaluates the suitability of different algorithm design paradigms before selecting the most appropriate solution. In addition, the correctness and time complexity of the chosen algorithm are analysed to demonstrate its effectiveness and computational efficiency for wildfire evacuation route planning.

2.0 Problem Definition

2.1 Geographical Setting

The story is set in Cedar Hollow, a small rural village surrounded by hills and a forest reserve. About 200 people live in the village, split across a few residential zones. A river runs along the south side, and one bridge is the only crossing point to the safe area outside the village.

2.2 Disaster Scenario: Wildfire

During a long dry season, a wildfire starts in the forest reserve on the village's northeast side. Strong wind pushes the fire toward the village. Within hours, smoke fills the area and the fire reaches the forest edge, right next to one of the village's main roads.

2.3 Damage and Impact

The forest-edge shortcut road catches fire from falling branches and heat, and becomes unusable. Zone B, the residential area closest to the forest, is cut off from its fastest route out.

Smoke also lowers visibility on nearby roads, slowing travel. The bridge becomes the only way to reach the shelter, since the shortcut is gone. If residents pick a route without knowing it's blocked, they lose time they don't have.

2.4 Importance of Optimal Algorithm

In a wildfire, minutes matter. People can't manually compare every route while the situation keeps changing (roads closing as the fire spreads) and panic makes decisions worse. A shortest-path algorithm can instantly tell each zone its fastest way out, and recalculate the moment a road is blocked. Guessing a route can cost lives; an algorithm guarantees the best one mathematically.

- **Predicting Performance Under Stress (Time Complexity):**
  In a disaster, the map layout changes rapidly as fire spreads and blocks roads. AAD teaches us that by using an array-based Dijkstra implementation, the time complexity is $O(V^2)$, where V is the number of vertices. For our 12 node evacuation network, this is computationally efficient and straightforward to implement. This mathematical guarantee ensures that even if the graph grows to include thousands of roads and intersections, the algorithm will calculate the path in milliseconds, allowing real-time re-routing before emergency vehicles deploy.

- **Resource Management (Space Complexity):**
  Emergency teams often deploy these systems on mobile units, satellite laptops, or local tablets in the field. AAD analysis proves the space complexity is $O(V + E)$ (to store the map and priority queue), ensuring the software can run flawlessly on lightweight, low-power field hardware without crashing due to memory overloads.

- **Ensuring System Reliability:**
  Algorithm Analysis and Design (AAD) principles ensure that the proposed solution produces consistent and reliable results. Since Dijkstra's Algorithm operates on graphs with non-negative edge weights, it guarantees the shortest evacuation route whenever a valid path exists. Additionally, blocked roads are excluded from the graph before the algorithm is executed, ensuring that only safe and accessible routes are considered during evacuation planning.

2.5 Goal and Expected Output

Goal:
To find the shortest path from every residential zone to the shelter, and update it automatically when a road is blocked by fire.

Expected Output:
For each starting zone, the sequence of locations to pass through and the total distance of the recommended route.

2.6 Illustration

Figure 2.1 illustrates zones, roads, the forest reserve, the wildfire zone, the river/bridge, and the shelter. The two dashed red roads near the forest are the ones cut off by the fire. Table 2.1 lists each node and its location, and Table 2.2 lists the distance and status of each road segment.

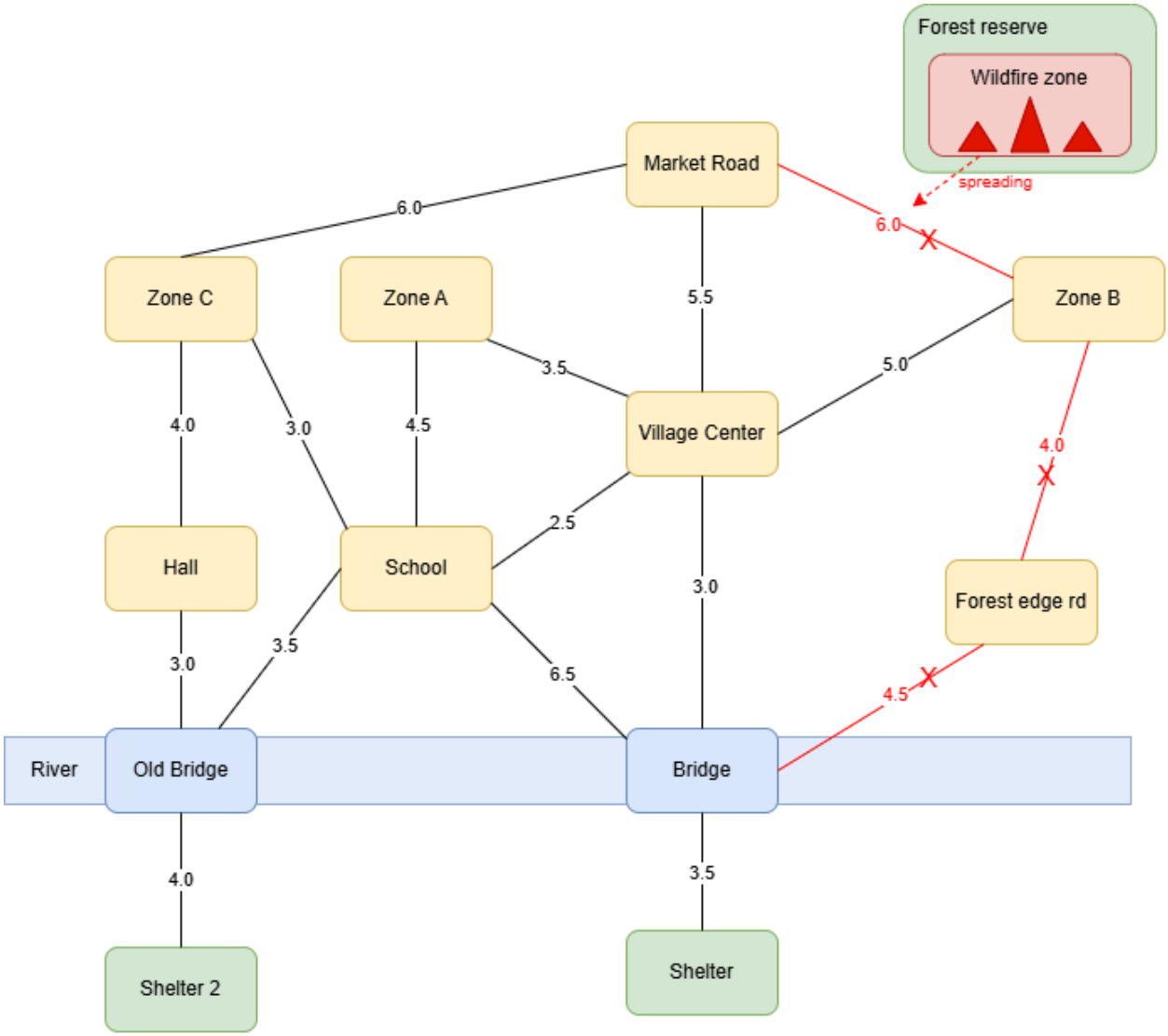


Figure 2.1: Cedar Hollow village road network, showing the wildfire zone and blocked roads to the shelter

Node	Location
A	Zone A (residential, north)

B	Zone B (residential, nearest forest)
C	Zone C (residential, west)
D	Village center
E	School
F	Forest edge road (junction)
G	Bridge
H	Shelter (destination)
I	Market Road
J	Community Hall
K	Old Bridge
L	Shelter 2 (destination)

Road	Distance (Km)	Status
A-D	3.5	open
A-E	4.5	open
B-D	5.0	open
B-F	4.0	blocked
B-I	6.0	blocked
C-E	3.0	open
C-I	6.0	open
C-J	4.0	open
D-E	2.5	open
D-I	5.5	open
D-G	3.0	open
E-G	6.5	open
E-K	3.5	open

F-G	4.5	blocked
G-H	3.5	open
J-K	3.0	open
K-L	4.0	open

Table 2.2: Road segments with distance and blockage status

3.0 Model Development

3.1 Graph Model

The Cedar Hollow evacuation network is modeled as a weighted undirected graph, denoted by ***G = (V, E)***, where V represents the set of locations (vertices) and E represents the road segments (edges). The graph model is constructed based on the road network illustrated in Figure 2.1, while the corresponding vertices and edge weights are defined in Table 2.1 and Table 2.2.

Each vertex represents a residential area, public facility, road junction, or evacuation shelter. Each edge represents a road connecting two locations, and the edge weight corresponds to the travelling distance in kilometres. Since roads can be travelled in both directions, the graph is modeled as an undirected weighted graph. Road segments that are blocked due to wildfire are excluded from the graph before the shortest path computation is performed.

3.2 Data Representation

The wildfire evacuation problem is represented using the following data structures during implementation.

Data	Data Type	Purpose
Graph	PriorityQueue(double[][])	Stores road distances between locations
Distance	int[]	Stores the current shortest distance from the source vertex
Visited	boolean[]	Indicates whether a vertex has been permanently processed
Parent	int[]	Stores the predecessor of

		each vertex for path reconstruction
Location Names	String[]	Stores the names of all locations

Table 3.1: Data representation table

3.3 Objective Function

The objective of the proposed evacuation system is to determine the shortest and safest evacuation route from a selected source location to the nearest reachable shelter.

The optimization objective can be expressed as

$$\min \sum_{(u,v) \in P} w(u,v)$$

where:

- P denotes the evacuation path.
- $w(u,v)$ denotes the distance between two connected locations.

The selected path must minimize the total travelling distance while avoiding blocked roads.

3.4 Constraints

The evacuation model operates under the following assumptions:

- All road distances are non-negative.
- Only open roads are considered during route computation.
- Blocked roads caused by the wildfire are excluded from the graph.
- Each evacuation starts from one selected source location.
- The destination is the nearest reachable shelter.
- At least one evacuation path exists from every source location to a shelter.

3.5 Example Input and Output

Example Input

- Source location: Zone A

Expected Output

- Recommended evacuation shelter.
- Shortest evacuation distance.

- Shortest evacuation route.
- Sequence of locations travelled from the source to the selected shelter.

4.0 Algorithm Specification

4.1 Dijkstra's Algorithm Justification

Dijkstra's algorithm was selected for this evacuation routing system due to several properties that make it well-suited for real-world disaster scenarios. The following points outline the key justifications for this choice.

1. **Representation of the Physical World as a Graph**
   The affected geographic region can be perfectly modeled as a directed or undirected graph, denoted as G = (V, E).

   - **Vertices (V):** Represent fixed physical locations such as villages (starting points), road intersections, checkpoints, and emergency shelters (destinations).
   - **Edges (E):** Represent the roads connecting these locations.

2. **Handling of Positive Weights (Distance and Risk)**
   Dijkstra's algorithm requires all edge weights to be non-negative ($w(u, v) \geq 0$). In a wildfire evacuation, edge weights represent a combination of travel time, physical distance, and risk factors. Because distance and time can never be negative, this algorithm is mathematically sound for this application.

3. **Single-Source Shortest Path (SSSP) Efficiency**
   Dijkstra's is a single-source shortest path algorithm. In our scenario, we need to find the optimal path from one specific isolated village (the source) to all potential safe zones/shelters (the destinations). Instead of calculating routes point-by-point, Dijkstra explores outward from the village, calculating the absolute shortest path to every accessible vertex until the safest, closest shelter is reached.

4. **Deterministic and Guaranteed Optimality**
   Unlike heuristic-based algorithms, which rely on approximations that might fail if a fire suddenly blocks a route, Dijkstra guarantees the mathematically shortest path based on the current known state of the map. In emergency logistics, guaranteeing optimality saves lives.

4.2 Review of Algorithmic Paradigms (Comparative Analysis)

Selecting the correct paradigm is a critical safety decision. Below is an evaluation of how different algorithmic paradigms perform under the strict operational constraints of an active wildfire.

- **Sorting Algorithms**

- **Core Concept**: Reordering a flat sequence of items based on a specific property.
- **Weaknesses**: Sorting algorithms completely lack a structural concept of network topology, nodes, or connectivity. Knowing which individual roads are short does not indicate whether those roads connect to form a valid path.
- **Paradigm Verdict**: Unsuitable**.** It cannot solve network-based routing problems.

- **Divide and Conquer (DAC)**

- **Core Concept:** Breaking a problem down into independent, non-overlapping sub-problems, solving them recursively, and combining the results.
- **Weaknesses:** Graph paths are highly interconnected and dependent on one another. If you arbitrarily cut an evacuation map in half to "divide" it, you break the continuous roads connecting the village to the shelters.
- **Paradigm Verdict:** Unsuitable. The structural dependencies of a road network defy clean top-down division.

- **Dynamic Programming (DP)**

- **Core Concept:** Breaking a problem down into overlapping sub-problems, computing solutions bottom-up, and storing results in a table (memoization) to avoid redundant recalculation.
- **Weaknesses**: This power comes with massive computational overhead. Algorithms like Floyd-Warshall run at O(|V|^3) time complexity. During a wildfire, computing paths between two safe, unaffected shelters is a waste of processing power.
- **Paradigm Verdict**: Poor / Overkill. It computes too much unnecessary data, making it too slow for real-time crisis updates.

- **Graph Algorithms (Traversal-Based / BFS)**

- **Core Concept:** Exploring a network structure systematically by moving from node to neighboring node layer by layer (Level-Order Exploration).
- **Weaknesses:** Basic graph traversal assumes all edge weights are identical (unweighted). In a wildfire, a short road might be completely blocked by thick smoke, while a longer highway is clear. BFS cannot see these weight variations.
- **Paradigm Verdict:** Inadequate. It is blind to real-world road hazards, travel times, and distances.

- **Greedy Paradigms (The Selected Solution: Dijkstra's**)

- **Core Concept:** Building a solution step-by-step by making the locally optimal ("greediest") choice at each stage, with the mathematical goal of arriving at a global optimum.
- **Strengths:** Because real-world road distances and travel times can never be negative ($w \geq 0$), a path can never get shorter by traversing additional edges. It handles variable road weights perfectly (allowing traffic and smoke penalties).
- **Paradigm Verdict**: Excellent**.** It delivers the variable weight-handling of complex DP algorithms while remaining highly deterministic.

Algorithm / Paradigm Class	**Core Design Paradigm**	**Edge Weight Constraints**	**Time Complexity (Standard)**	**Suitability for Wildfire Evacuation**
Dijkstra's Algorithm	Greedy Approach	Must be strictly non-negative ($w \geq 0$)	O(|V|^2) with Weight Matrix & Array-based Queue)	Excellent. Guarantees the absolute shortest path and optimally handles dynamic road hazard penalties/weights.
Bellman-Ford Algorithm	Dynamic Programming	Can handle negative weights and detect negative cycles	O(|V| x |E|)	Poor. Designed to hunt for negative weight cycles. Since physical road lengths/times cannot be negative, its slow speed is a liability during a crisis.
Floyd-Warshall Algorithm	Dynamic Programming	Can handle negative weights but no negative cycles	O(|V|^3)	Poor. Computes paths between all pairs of nodes. This processes massive unnecessary data (e.g., paths between two safe shelters), causing execution delays.

Breadth-First Search (BFS)	Graph Traversal	Unweighted edges only (all edge weights implicitly = 1)	O (|V| + |E|)	Inadequate. Assumes all roads have uniform travel costs. Cannot factor in reality where a short road is choked with heavy smoke.
Sorting Algorithms	Element Ranking	N/A (Operates on isolated data lists)	O(N log N)	Unsuitable. Only capable of sorting independent values. It completely lacks a concept of network topology or connectivity.
Divide and Conquer (DAC)	Top-Down Decomposition	N/A (Operates on non-overlapping subsets)	Varies by sub-problem structural splits	Unsuitable. Graph paths are highly interconnected. Arbitrarily splitting a physical map into separate halves breaks the continuous roads needed.

The Greedy paradigm works by making the locally optimal choice at each stage with the hope of finding a globally optimal solution. Dijkstra selects the nearest unvisited intersection at every step. Because the edge weights (distances/risks) are strictly positive, a path can never get "shorter" by adding more roads. Thus, the greedy choice is mathematically guaranteed to yield the globally shortest evacuation route.

4.3 Algorithm Specification

This specification defines the exact logical blueprint, data structures, and rules governing our evacuation system, based on a matrix-and-array implementation.

A. Input Data Requirements (Preconditions)

The algorithm requires four primary inputs before execution can begin:

- **The Graph (G):** A 2D **Weight Matrix (Adjacency Matrix)** of size |V| x |V| representing the road network, where rows and columns represent vertices (locations), and cells store edge weights.
- **Edge Weights (w):** Positive numerical values representing travel time or distance stored in the matrix. If no direct road connects two intersections, or if a road is completely blocked by fire, its weight is set to Infinity.
- **The Source Node (s):** The specific integer ID of the isolated village requiring immediate evacuation.
- **Target Set (T):** A list of available, active emergency shelters.

B. Expected Output Data

Upon successful execution, the algorithm must return:

- **Optimal Path Cost (Scalar Value):** A single number representing the absolute shortest distance (e.g., 14.5 KM) or time to the nearest shelter.
- **Route Sequence (Ordered List):** An ordered vector of nodes showing the exact path to take (e.g., [Village A -> Intersection 4 -> Shelter 2]).
- **Failure Flag:** If a village is completely trapped by fire, the algorithm must cleanly return a NO_SAFE_ROUTE status instead of crashing.

C. Core Operational Logic & Math Invariant

The algorithm utilizes the Greedy paradigm.

- **The Loop Invariant:** The core rule is that the moment a node is marked as "visited," its shortest path from the source is mathematically finalized. Because all road weights are positive ($w \geq 0$), the algorithm never has to waste time backtracking.
- **Edge Relaxation:** The algorithm continuously updates tentative distances to neighboring nodes using the formula:

$$\textbf{dist[u] + w(u, v) < dist[v] then dist[v] = dist[u] + w(u, v)}$$

- **u:** The intersection or location you are currently standing at.
- **v:** A neighboring intersection you are looking toward.
- **dist[v]:** The current best time/distance you have on record to get from your starting village to location v. (If you haven't found a path to it yet, this starts at Infinity)
- **dist[u]: T**he total time/distance it took to get from your starting village to where you are standing right now (u).
- **w(u, v):** The weight (travel time, physical distance, or fire risk penalty) of the direct road connecting u to v**.**

D. Required Data Structures

To implement this specific version, the following components are utilized:

- **Weight Matrix (Adjacency Matrix):** A 2D array of size |V| x |V|. It allows O(1) constant-time lookup to check the exact weight/distance between any two nodes u and v.
- **Min-Priority Queue (Q):** Implemented via a **Standard Unordered Array** (or a boolean tracking array visited[V]). To find the next closest unvisited node, the system performs a linear scan across the array, taking O(|V|) time per extraction.
- **Distance Array:** A 1D array of size |V| that tracks the current shortest distance from the source to every node. Initialize all nodes to infinity and the source to 0.
- **Predecessor Array (prev):** A 1D array of size |V| that records the "breadcrumbs" of the path. This allows the code to trace the route backward from the final shelter to the village to build the final directions.

E. Performance Boundaries (Complexity Constraints)

- **Time Complexity:** Tightly bounded at **O(|V|^2)**. In each of the |V| iterations, a linear scan of size O(|V|) is performed to locate the minimum distance vertex from the array, and all |V| columns in the weight matrix row are evaluated for relaxation.
- **Space Complexity:** Bounded at **O(|V|^2)** due to the storage requirements of the |V| x |V| Weight Matrix, ensuring predictable memory allocation on field systems.

5.0 Algorithm Design

5.1 Flowchart

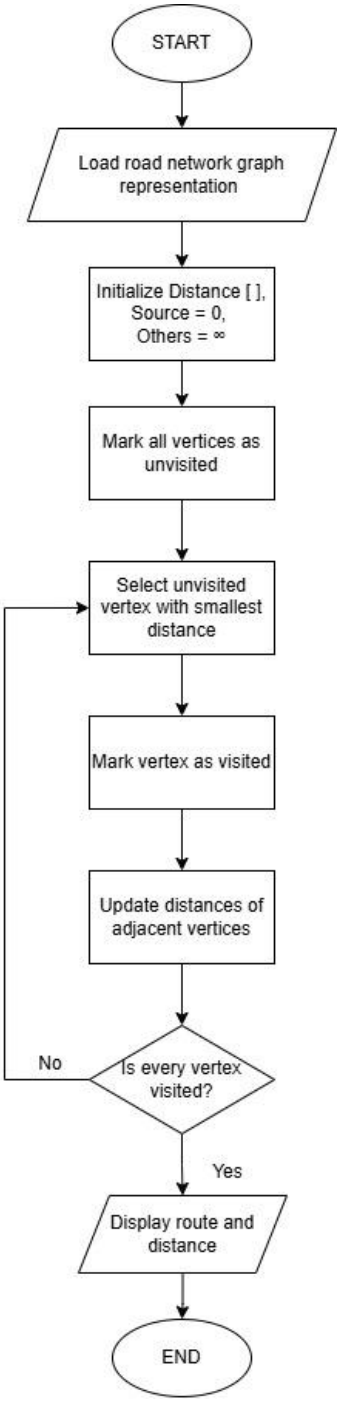


Figure 5.1: Flowchart of Dijkstra Algorithm

5.2 Pseudocode

```
DIJKSTRA(Graph G, Source s)

FOR each vertex v in G
   distance[v] ← ∞
   visited[v] ← false
   parent[v] ← null

distance[s] ← 0

FOR i ← 1 to |V| DO

   u ← unvisited vertex with minimum distance

   visited[u] ← true

   FOR each neighbor v of u DO

      IF visited[v] = false AND distance[u] + weight(u,v) < distance[v] THEN

         distance[v] ← distance[u] + weight(u,v)
         parent[v] ← u

      END IF
   END FOR
END FOR

RETURN distance[], parent[]
```

5.3 Output Example

```
C:\Users\Azib\.jdks\openjdk-23.0.2\bin\java.exe "-javaagent:
Enter the Starting zone number(A = 0, B = 1, C = 2): 0
The shortest path from node :
A to A is 0
A to B is 85
A to C is 75
A to D is 35
A to E is 45
A to F is -1
A to G is 65
A to H is 100
A to I is 90
A to J is 110
A to K is 80
A to L is 120
Closest destination: H
Distance: 10.0 km
Path: A -> D -> G -> H
Time taken in millisecond: 555900 ns
```

Figure 5.2: Output of the code running for starting zone A

```
C:\Users\Azib\.jdks\openjdk-23.0.2\bin\java.exe "-javaag
Enter the Starting zone number(A = 0, B = 1, C = 2): 1
The shortest path from node :
B to A is 85
B to B is 0
B to C is 105
B to D is 50
B to E is 75
B to F is -1
B to G is 80
B to H is 115
B to I is 105
B to J is 140
B to K is 110
B to L is 150
Closest destination: H
Distance: 11.5 km
Path: B -> D -> G -> H
Time taken in millisecond: 643400 ns
```

Figure 5.3: Output of the code running for starting zone B

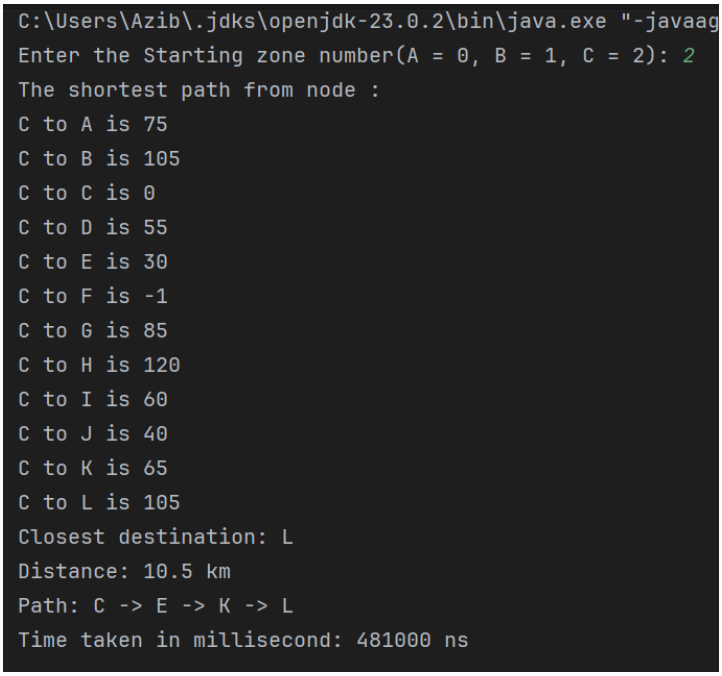


Figure 5.4: Output of the code running for starting zone C

6.0 Correctness Analysis

Correctness of Dijkstra's Algorithm

The proposed wildfire evacuation system uses Dijkstra's Algorithm to determine the shortest evacuation route from a residential or village to the nearest shelter. The correctness of the algorithm relies on the greedy-choice property (Cormen et al., 2022). Initially, the source village is assigned a distance of 0, while all other vertices are assigned infinity. During each iteration, the algorithm selects the unvisited vertex with the smallest tentative distance.

Because all road distances in the evacuation network are non-negative, once a vertex is selected as the minimum-distance unvisited vertex, its shortest distance from the source is guaranteed and cannot be improved by any future path. The algorithm then performs edge relaxation by updating the distances of adjacent vertices whenever a shorter route is discovered. This process continues until all vertices have been visited.

Consequently, every vertex receives the minimum possible distance from the source village or residential, and every parent vertex recorded represents part of a shortest path. As a result, the reconstructed route from the shelter back to the source forms the optimal evacuation route. Therefore, Dijkstra's Algorithm always produces the shortest evacuation route in a weighted graph with non-negative edge weights.

Proof of Correctness

Assume that vertex u is selected as the unvisited vertex with the smallest tentative distance.

Suppose a shorter path to u exists.

That path must pass through another unvisited vertex v.

However:

$$distance(v) \geq distance(u)$$

because u was chosen as the minimum-distance unvisited vertex.

Since all edge weights are non-negative:

$$distance(v) + weight(v,u) \geq distance(u)$$

which contradicts the assumption that a shorter path exists.

Therefore, the distance assigned to u is optimal when selected.

Therefore, the distance assigned to u is optimal when selected. By repeating this process for all vertices, the algorithm correctly computes the shortest path from the source village or residential to reachable shelter (Cormen et al., 2022).

7.0 Time Complexity Analysis

The proposed solution uses the standard array-based implementation of Dijkstra's Algorithm.

Let:

$$V = Number\ of\ Vertices$$
$$E = Number\ of\ Edges$$

7.1 Best Case Analysis

For each iteration, the algorithm searches all vertices to identify the unvisited vertex with the minimum distance.

This operation requires

$$O(V)$$

time.

Since this selection process is performed for all vertices:

$$T(V) = V \times O(V)$$

Therefore:

$$T(V) = O(V^2)$$

Hence, Best Case Complexity:

$$O(V^2)$$

7.2 Average Case Analysis

In the average case, the algorithm still performs:

- V minimum-distance searches
- Multiple edge relaxations

The dominant operation remains the repeated minimum search.

Hence:

$$T(V) = V \times O(V)$$

$$T(V) = O(V^2)$$

Average Case Complexity:

$$O(V^2)$$

7.3 Worst Case Analysis

The worst case occurs when every vertex and every edge must be processed before the shortest paths are determined.

The algorithm performs:

$$V$$

iterations.

During each iteration:

$$O(V)$$

comparisons are required to locate the next minimum-distance vertex.

Therefore:

$$T(V) = V \times O(V)$$

$$T(V) = O(V^2)$$

Worst Case Complexity:

$$O(V^2)$$

7.4 Growth of Function

The growth of the algorithm is quadratic.

$$f(V) = V^2$$

As the number of villages, shelters, and road junctions increases, the execution time grows proportionally to the square of the number of vertices.

For example:

Vertices (V)	**Growth (V^2)**
10	100
20	400
50	2500
100	10000

This demonstrates the quadratic growth rate of the standard implementation of Dijkstra's Algorithm.

8.0 Conclusion

This project successfully demonstrates the application of Dijkstra's Algorithm in solving the wildfire evacuation route planning problem. By modelling the Cedar Hollow road network as a weighted graph, the system is able to determine the shortest evacuation route from different source locations to the nearest available shelter while avoiding blocked roads caused by the wildfire. The Java implementation verifies that the algorithm consistently produces the optimal evacuation route based on the given road network.

Among the algorithm paradigms evaluated, Dijkstra's Algorithm was found to be the most suitable because it efficiently solves the single-source shortest path problem for graphs with non-negative edge weights. The correctness analysis confirms that the greedy strategy guarantees the shortest path, while the time complexity analysis shows that the standard array-based implementation operates in $O(V^2)$ time, making it appropriate for small to medium sized evacuation networks.

Overall, the proposed system demonstrates how graph algorithms can support emergency evacuation planning and improve decision-making during disaster situations. Future enhancements may include incorporating real-time traffic information, dynamic road conditions, and Geographic Information System (GIS) integration to further improve the accuracy and practicality of the evacuation planning system.

9.0 References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Elaf Jirjees Dhulkefl, Akif Durdu, & Hakan Terzioğlu. (2020). DIJKSTRA ALGORITHM USING UAV PATH PLANNING. Selcuk University Journal of Engineering, Science and Technology, 8, 92–105. https://doi.org/10.36306/konjes.822225

GeeksforGeeks. (2026, January 19). *Introduction to Divide and Conquer Algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/introduction-to-divide-and-conquer-algorithm/

GeeksforGeeks. (2024, January 17). *Graph Algorithms*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/

10.0 Appendices

Initial Project Plan

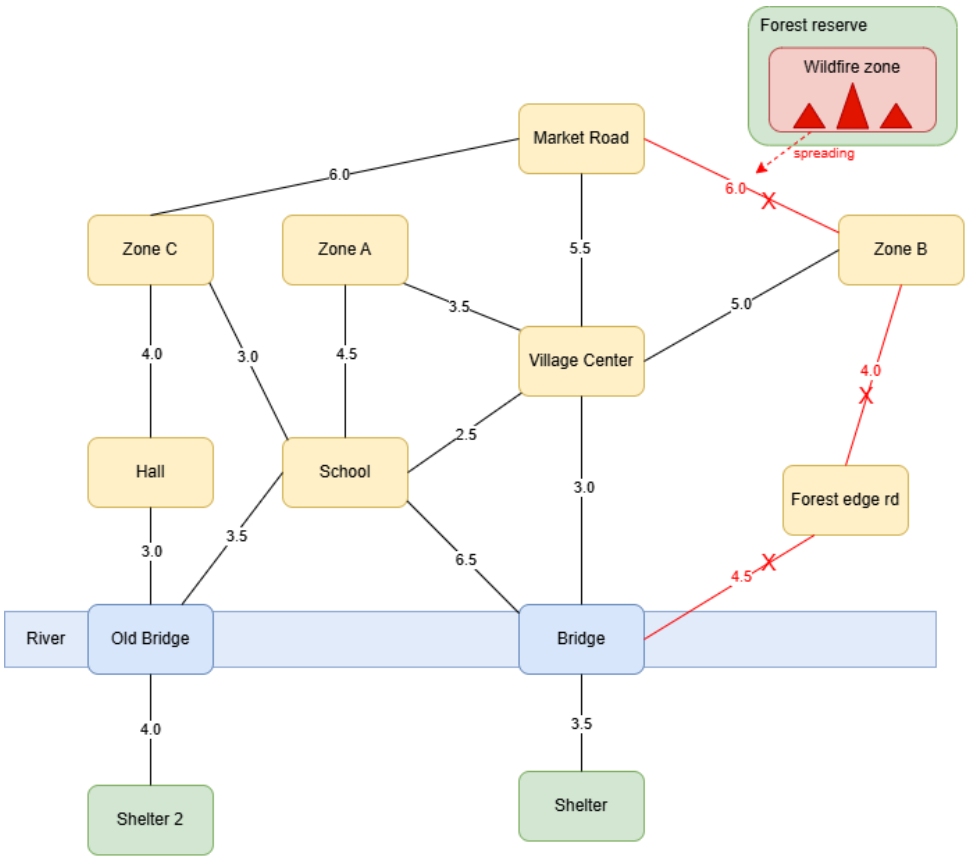
<table>
<tr><td>Group Name</td><td>Group 1</td></tr>
<tr><td>Members</td><td>
<table>
<tr><th>Name</th><th>Email</th><th>Phone number</th></tr>
<tr><td>NUFAIL AZRI BIN AZMI</td><td>224872@student.upm.edu.my</td><td>011-37781974</td></tr>
<tr><td>DZAKY KEENAN IBRAHIM</td><td>218430@student.upm.edu.my</td><td>019-9006096</td></tr>
<tr><td>SUJENIZSWARAN A/L RAJAH</td><td>225284@student.upm.edu.my</td><td>011-12366434</td></tr>
<tr><td>MUHAMMAD AZIB BIN AZMI</td><td>223796@student.upm.edu.my</td><td>012-6002857</td></tr>
</table>
</td></tr>
<tr><td>Problem scenario description</td><td>A wildfire has spread near Cedar Hollow village, causing several roads to become blocked. Residents from multiple residential zones must evacuate safely to the nearest available shelter. The system aims to determine the shortest evacuation route while avoiding blocked roads using Dijkstra's Algorithm.</td></tr>
<tr><td>Why it is important</td><td>Efficient evacuation planning is essential during wildfire emergencies to reduce evacuation time, minimize traffic congestion, and improve public safety. Selecting the shortest available route helps emergency responders and residents reach evacuation shelters more quickly.</td></tr>
<tr><td>Problem specification</td><td>● Model the road network as a weighted graph.
● Vertices represent locations within the village.
● Edges represent road segments.
● Edge weights represent travelling distance (km).
● Blocked roads are excluded from the graph.
● The system finds the shortest route from a selected source to the nearest shelter.</td></tr>
<tr><td>Potential solutions</td><td>● Sorting Algorithms: not suitable
● Divide and Conquer: not suitable</td></tr>
</table>

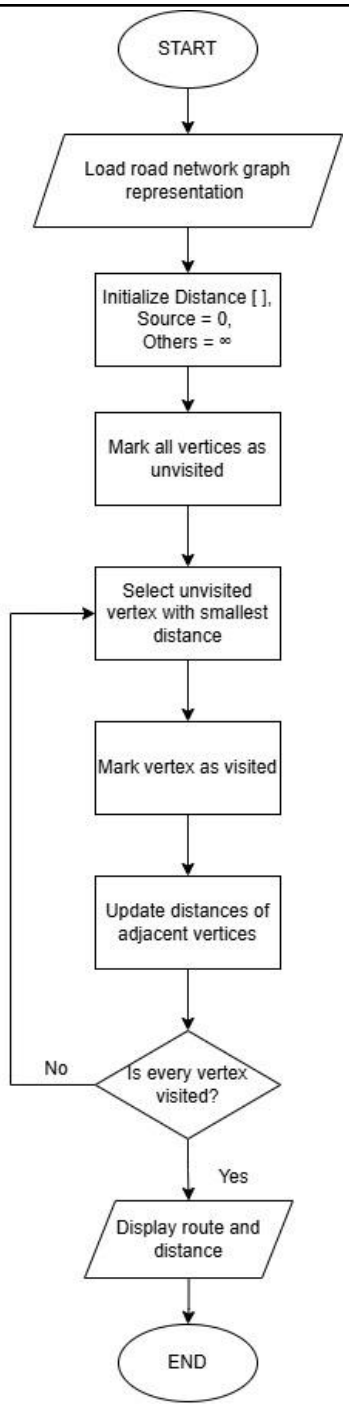
	● Dynamic Programming: possible but unnecessary for a single-source shortest path problem ● Greedy Algorithm (Dijkstra's Algorithm): Chosen solution ● Graph Traversal (BFS/DFS): Not suitable because these algorithms ignore edge weights.
Sketch (framework, flow, interface)	● Road network diagram of Cedar Hollow ● Graph representation (vertices and weighted edges) ● Dijkstra Algorithm flowchart ● Console-based Java interface for selecting evacuation source and displaying the recommended shelter

Project Proposal Refinement

Group Name	Group 1
Members	<table><tr><th>Name</th><th>Role</th></tr><tr><td>Nufail</td><td>Project leader</td></tr><tr><td>Dzaky</td><td>System Analyst</td></tr><tr><td>Sujen</td><td>Algorithm Designer</td></tr><tr><td>Azib</td><td>Java Developer & Tester</td></tr></table>
Problem statement	Wildfires can block important road segments and delay evacuation. Residents require a reliable system that can recommend the safest and shortest evacuation route to an available shelter. Manual route selection may lead to longer travel times and increased risk.
Objectives	● Model the evacuation network as a weighted graph. ● Implement Dijkstra's Algorithm in Java. ● Determine the shortest evacuation route from different source locations. ● Recommend the nearest available shelter. ● Analyse the correctness and time complexity of the algorithm.
Expected output	● Recommended evacuation shelter. ● Shortest evacuation distance. ● Shortest evacuation route.

	• Console output displaying the selected path.
Problem scenario description	The project simulates a wildfire evacuation system for Cedar Hollow village. Residential zones, public facilities, road junctions, and shelters are represented as graph vertices. Road distances are represented by weighted edges, while blocked roads are removed before route computation.
Why it is important	Rapid route planning helps residents evacuate efficiently during emergencies while reducing travel distance and avoiding inaccessible roads.
Problem specification	• Weighted undirected graph. • Non-negative edge weights. • Multiple possible evacuation sources. • Two available shelters. • Blocked roads excluded from computation. • Output the shortest route and distance.
Potential solutions	After evaluating Sorting, Divide and Conquer, Dynamic Programming, Greedy, and Graph algorithms, Dijkstra's Algorithm was selected because it efficiently computes the shortest path in weighted graphs with non-negative edge weights.

Sketch (framework, flow, interface)	 Road Network Diagram


START
Load road network graph representation
Initialize Distance [ ], Source = 0, Others = ∞
Mark all vertices as unvisited
Select unvisited vertex with smallest distance
Mark vertex as visited
Update distances of adjacent vertices
Is every vertex visited?
No
Yes
Display route and distance
END

Flowchart

<table>
<tr><td>Methodology</td><td>

Milestone	Time
Scenario selection and requirement analysis	Week 10
Literature review and algorithm selection	Week 11
System modelling, flowchart and pseudocode	Week 12
Java implementation and debugging	Week 13
Correctness analysis, complexity analysis, online portfolio and presentation preparation	Week 14

</td></tr>
</table>

Project Progress (Week 10 – Week 14)

<table>
<tr><td>Milestone 1</td><td></td></tr>
<tr><td>Date (week)</td><td>Week 10–11</td></tr>
<tr><td>Description/ sketch</td><td>

- Brainstorm project ideas.
- Selected wildfire evacuation scenario.
- Conducted literature review.
- Compared algorithm paradigms.
- Selected Dijkstra's Algorithm.
- Designed initial road network and graph model.

</td></tr>
<tr><td>Role</td><td>

Name	Role
Nufail	Prepared problem scenario and illustrations
Dzaky	Developed graph model and documentation

</td></tr>
</table>

	Sujen	Designed flowchart and pseudocode
	Azib	Planned Java implementation

Milestone 2	
Date (Wk)	Week 12–14
Description/ sketch	● Completed Java implementation. ● Tested different evacuation sources. ● Verified shortest routes. ● Analysed correctness and time complexity. ● Prepared online portfolio. ● Finalised report and presentation slides.
Role	

Name	**Role**
Nufail	Finalised report writing and presentation slides
Dzaky	Edited documentation and references
Sujen	Completed correctness proof,

		complexity analysis and presentation
	Azib	Completed Java program, testing and GitHub/portfolio